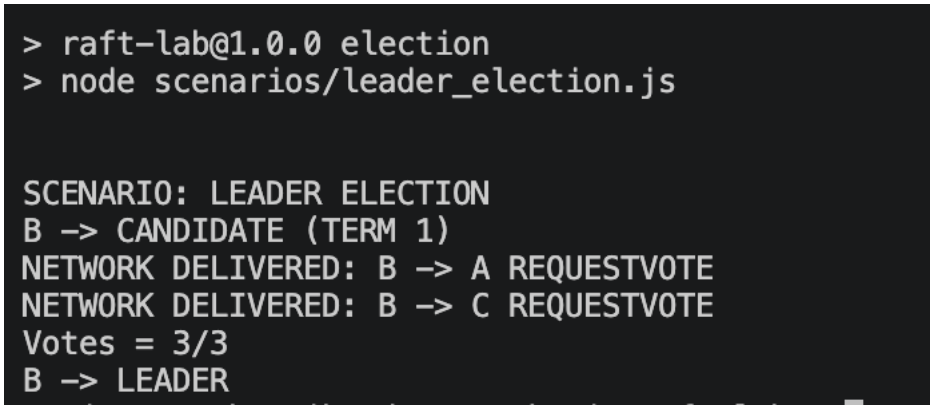
Part1:



Explanation：

The cluster contains three nodes: A, B, and C, which initially start as followers. The scenario selects B to become a candidate. B increases its term to 1, votes for itself, and sends RequestVote messages to A and C. The simulator counts both online peers as granting votes, producing a total of 3/3.

A majority of two votes is required, so B becomes the leader for term 1. This establishes leadership before log replication begins. However, the simulator simplifies elections: B is selected directly, and votes are counted automatically without the full voting checks required by Raft.

Reflection Questions

1. Why must a leader be elected before processing requests?

Raft uses a leader to coordinate client commands, determine their order in the replicated log, and manage replication to followers. Electing a leader first establishes which node is responsible for coordinating these operations.

2. Why is a majority required?

A majority ensures that any two election quorums overlap. Together with the rule that each node grants at most one vote per term, this prevents two candidates from both winning an election in the same term. In this three-node cluster, the required majority is two.

3. What prevents multiple leaders from being elected simultaneously?

In full Raft, a candidate must receive a majority of votes, and each node can vote for only one candidate per term. Because any two majorities overlap, two candidates cannot both

become leaders in the same term. Randomised election timeouts help reduce split votes. The supplied simulator does not fully implement these safeguards, so this run demonstrates one successful election rather than verifying protection against competing candidates.

4. What role do terms play in the election process?

Terms act as logical election numbers. A candidate increments its term when starting an election. Nodes use term numbers to distinguish current authority from outdated authority. In full Raft, a node that discovers a higher term updates its term and becomes a follower. In this experiment, B advances from term 0 to term 1 and becomes the leader for that term.

Part2:

```
> raft-lab@1.0.0 heartbeats
> node scenarios/heartbeats.js


SCENARIO: HEARTBEATS
B -> CANDIDATE (TERM 1)
NETWORK DELIVERED: B -> A REQUESTVOTE
NETWORK DELIVERED: B -> C REQUESTVOTE
Votes = 3/3
B -> LEADER

Roles before heartbeats:
A: FOLLOWER
B: LEADER
C: FOLLOWER

Sending heartbeats:
NETWORK DELIVERED: AppendEntries(term=1, entries=[])
NETWORK DELIVERED: AppendEntries(term=1, entries=[])

Roles after heartbeats:
A: FOLLOWER
B: LEADER
C: FOLLOWER
```

Explanation：

B becomes the leader for term 1 after receiving three votes. It then sends two AppendEntries messages, one for each follower. Both contain an empty entries list, indicating heartbeats rather than new log entries. Before and after the heartbeat call, A and C remain followers, while B remains leader.

In full Raft, valid heartbeats maintain leadership by resetting followers' election timers and preventing unnecessary elections. However, this simulator only prints messages and does

not implement heartbeat reception or election timers. The output therefore demonstrates heartbeat structure and unchanged roles, but does not prove that timer resets occurred.

Reflection Questions

1. Why are heartbeats necessary?

Heartbeats let followers know that the leader is still communicating. They reset election timers, preventing unnecessary elections and helping maintain stable leadership.

2. Why does Raft use AppendEntries for both replication and heartbeats?

Both operations communicate the leader's authority and share term and log-consistency checks. AppendEntries carries log entries during replication and an empty entries list when used as a heartbeat.

3. What would happen if heartbeats stopped arriving?

Without valid leader communication, followers' election timers eventually expire. A follower then becomes a candidate, increments its term, and requests votes. Electing a replacement leader requires a majority. This simulator does not automatically model that process.

4. Why do followers reset their election timers?

Resetting the timer postpones elections while the current leader remains reachable. This reduces unnecessary leadership changes. Messages containing an outdated term should not reset the timer.

Part3:

```
> raft-lab@1.0.0 leader-failure
> node scenarios/leader_failure.js

B -> LEADER

====================
LEADER FAILURE
====================

Leader B fails
B OFFLINE

Election timeout
A -> CANDIDATE (TERM 1)
NETWORK DELIVERED: A -> C REQUESTVOTE
Votes = 2/3
A -> LEADER
```

Explanation：

B is initially assigned the leader role and then taken offline. The scenario prints an election timeout message and directly starts an election on A. A becomes a candidate for term 1, votes for itself, and sends a RequestVote message to C. The simulator counts C's vote, giving A two votes out of three. This meets the required majority, so A becomes the new leader.

The output demonstrates replacement leadership with one unavailable node. However, timeout detection is scripted, voting checks are simplified, and B's stored leader role is not cleared. The scenario therefore does not fully verify safe leader replacement.

Reflection Questions

1. How do followers detect leader failure?

In full Raft, followers suspect leader failure when they receive no valid leader communication before their election timers expire. They cannot directly distinguish a crashed leader from a communication failure. This scenario prints a timeout message rather than implementing a timer.

2. Why is a new term created?

A new term identifies a new election and distinguishes it from earlier elections. A increments its own term from 0 to 1 when becoming a candidate. Term numbers help nodes recognise outdated leadership and messages.

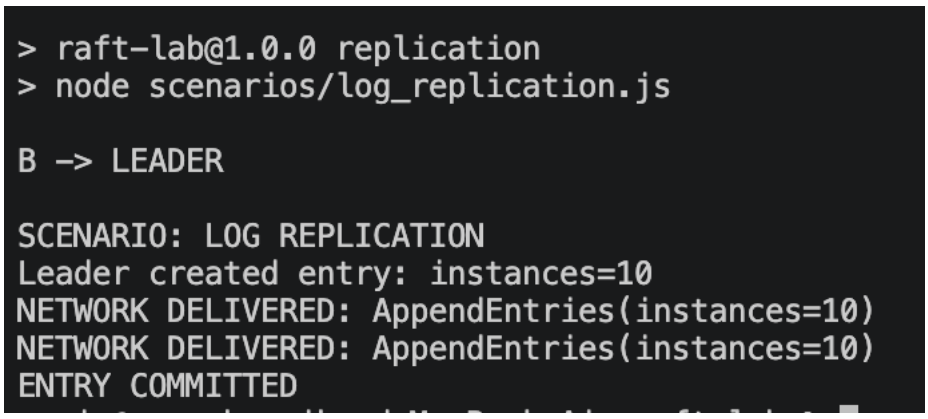
3. Why does the old leader lose authority?

An old leader cannot commit new entries without majority support. In full Raft, it steps down when it discovers a higher term. Here, B is taken offline, but the code does not update its stored role or demonstrate its response to a higher term.

4. How does Raft ensure that only one leader remains active?

Full Raft permits at most one elected leader per term through majority voting and one vote per node per term. A stale leader may temporarily believe it is still leader, but cannot independently commit conflicting entries. Higher-term messages cause it to step down. This simulator does not fully implement these protections.

Part4:



Explanation：

The scenario submits the command instances=10 directly to leader B. B creates a log entry and stores it in its own log. The simulator then appends the entry to the logs of online followers A and C, printing two AppendEntries messages. It counts three acknowledgements, including the leader's own copy. Since a three-node cluster requires a majority of two, it prints ENTRY COMMITTED.

This illustrates how majority replication supports commitment and resilience to node failure. However, acknowledgements are counted automatically, and logs exist only in memory. The experiment therefore demonstrates simulated commitment, not durable storage that survives process restarts.

Reflection Questions

1. Why does only the leader accept client requests?

The leader coordinates updates and establishes a consistent order for commands in the replicated log. Followers normally redirect client write requests to the leader rather than independently deciding their order.

2. What exactly is being replicated?

The replicated object is a log entry containing an index, a term, and a command. In this scenario, the entry contains index 1, term 0, and the command instances=10. The simulator appends it to each online follower's log.

3. When does an entry become committed?

In this simulator, an entry is reported as committed when the acknowledgement count reaches a majority. In full Raft, a leader can advance commitment through an entry from its current term once that entry is replicated on a majority; preceding entries then become committed as well.

4. Why does a majority acknowledgement matter?

A majority means the entry is stored on enough nodes to overlap with any future election majority. Combined with Raft's voting and log-consistency rules, this protects committed history when leadership changes. The simulator illustrates the majority count but does not implement all these safeguards.

Part5:

```
> raft-lab@1.0.0 follower-failure
> node scenarios/follower_failure.js

A -> LEADER
C OFFLINE

====================
FOLLOWER FAILURE
====================

C is offline
Leader created entry: instances=10
NETWORK DELIVERED: AppendEntries(instances=10)
ENTRY COMMITTED

LESSON:
Raft tolerates minority failures.
```

Explanation：

Node A is assigned the leader role, while follower C is taken offline. A creates a log entry containing instances=10 and replicates it to the remaining online follower, B. The simulator counts two acknowledgements, including A's own copy. This meets the majority requirement of two in a three-node cluster, so the entry is committed.

Compared with Part 4, replication messages decrease from two to one, and counted acknowledgements decrease from three to two. Commitment still succeeds because Raft requires a majority rather than every node. This demonstrates continued progress despite one follower failure, although acknowledgements and commitment are simplified in this simulator.

Reflection Questions

1. Why does the cluster continue functioning?

A and B remain online and form the required majority of two. The leader can therefore replicate the entry to enough nodes for the simulator to report it as committed, despite C being unavailable.

2. How many failures can this deployment tolerate?

This three-node deployment can tolerate one unavailable node while retaining a majority. If two nodes become unavailable, only one remains, which is insufficient for new elections or commits. Progress also requires communication between the surviving nodes.

3. Which nodes are essential for progress?

In this scenario, A and B are both necessary because they are the only remaining majority. More generally, Raft needs a functioning leader and a communicating majority, rather than any permanently indispensable node. A failed leader can be replaced through an election.

4. Why does Raft use quorums?

Quorums allow progress without requiring every node to respond. Majority quorums overlap, which, together with Raft's voting and log-consistency rules, helps preserve committed history across failures and leadership changes.

Part6:

```
> raft-lab@1.0.0 majority-lost
> node scenarios/majority_lost.js

A -> LEADER
C OFFLINE
D OFFLINE
E OFFLINE

====================
MAJORITY LOST
====================
Majority Needed: 3
Online Nodes: 2

NO MAJORITY AVAILABLE
Cannot elect leader
Cannot commit entries

LESSON:
Safety is preserved by sacrificing availability.
```

Explanation：

The cluster contains five nodes, so a majority requires three. A is initially assigned the leader role, while C, D, and E are taken offline. Only A and B remain available, leaving two online nodes. Because this is below the required majority, the scenario reports that no new leader can be elected and no new entries can be committed.

Raft stops making progress without a quorum to protect consistency rather than risk conflicting decisions. However, this scenario only checks the online-node count and prints these conclusions. It does not actually attempt an election or replication, and A retains its assigned leader role.

Reflection Questions

1. What requirement has been violated?

The requirement for an available, communicating majority is no longer satisfied. This five-node cluster needs three participating nodes, but only two remain online. The quorum is based on the configured cluster size, not just the surviving nodes.

2. Why can no leader be elected?

A candidate needs at least three votes to win an election. With only two nodes available, it cannot obtain enough votes, even including its own. A’s existing leader role does not mean a new election can succeed.

3. Why can no new entries be committed?

New entries cannot be replicated to the required majority of three nodes. Even if A continues acting as leader and stores entries locally, copies on A and B alone are insufficient for commitment.

4. Which property is being protected?

Raft protects safety, particularly the consistency of committed history. Refusing to commit without a majority prevents a minority from independently deciding updates that could conflict with another majority. Progress becomes unavailable until a communicating quorum is restored.

Part7:

```
> raft-lab@1.0.0 partition
> node scenarios/network_partition.js


====================
NETWORK PARTITION
====================

Leader communicating normally
NETWORK DELIVERED: AppendEntries(term=1)

Partition occurs
NETWORK PARTITION ACTIVE
NETWORK DROPPED: AppendEntries(term=1)
NETWORK DROPPED: AppendEntries(instances=10)

Partition repaired
NETWORK PARTITION REPAIRED
NETWORK DELIVERED: AppendEntries(term=1)

LESSON:
Communication failure affects availability.
```

```
PART 7: NETWORK PARTITION AND MAJORITY AGREEMENT
Configured nodes: 5; majority required: 3

Before partition:
A -> CANDIDATE (TERM 1)
DELIVERED: A -> B RequestVote
DELIVERED: A -> C RequestVote
DELIVERED: A -> D RequestVote
DELIVERED: A -> E RequestVote
Votes for A: 5/5; required: 3
A -> LEADER
DELIVERED: A -> B AppendEntries(instances=10)
DELIVERED: A -> C AppendEntries(instances=10)
DELIVERED: A -> D AppendEntries(instances=10)
DELIVERED: A -> E AppendEntries(instances=10)
A: acknowledgements 5/5; COMMITTED: instances=10

PARTITION ACTIVE: [A, B] | [C, D, E]
All five nodes remain online. The majority threshold stays at 3.

Majority side elects a replacement leader:
C -> CANDIDATE (TERM 2)
DROPPED: C -> A RequestVote
DROPPED: C -> B RequestVote
DELIVERED: C -> D RequestVote
DELIVERED: C -> E RequestVote
Votes for C: 3/5; required: 3
C -> LEADER
```

```
Minority side tries to commit using its old leader:
DELIVERED: A -> B AppendEntries(instances=20)
DROPPED: A -> C AppendEntries(instances=20)
DROPPED: A -> D AppendEntries(instances=20)
DROPPED: A -> E AppendEntries(instances=20)
A: acknowledgements 2/5; NOT COMMITTED: instances=20

Majority side tries to commit using its new leader:
DROPPED: C -> A AppendEntries(instances=30)
DROPPED: C -> B AppendEntries(instances=30)
DELIVERED: C -> D AppendEntries(instances=30)
DELIVERED: C -> E AppendEntries(instances=30)
DROPPED: C -> A Commit notification
DROPPED: C -> B Commit notification
C: acknowledgements 3/5; COMMITTED: instances=30

RESULT:
A: term=1, role=LEADER, committed=["instances=10"]
B: term=1, role=FOLLOWER, committed=["instances=10"]
C: term=2, role=LEADER, committed=["instances=10","instances=30"]
D: term=2, role=FOLLOWER, committed=["instances=10","instances=30"]
E: term=2, role=FOLLOWER, committed=["instances=10","instances=30"]
PASS: Minority update stayed uncommitted; majority update committed.
Scope: scripted elections and synchronous delivery; no partition-healing simulation.
```

Explanation：

The original scenario demonstrates message loss and restored delivery. The additional experiment divides five online nodes into a minority partition containing A and B and a majority partition containing C, D, and E. The majority requirement remains three. C receives three votes and becomes leader for term 2. The old leader A obtains only two

acknowledgements for instances=20, so that entry remains uncommitted. C obtains three acknowledgements for instances=30 and commits it. This demonstrates that minority leadership cannot independently commit conflicting updates. However, elections are triggered manually, communication is synchronous, and the additional scenario does not simulate log reconciliation after the partition heals.

Reflection Questions

1. Why is split-brain dangerous?

Split-brain is dangerous when disconnected groups independently commit conflicting updates. This can create incompatible histories and inconsistent application state. Raft prevents independent commitment by requiring majority agreement.

2. Which side remains authoritative?

The partition containing C, D, and E can continue committing entries because it contains the required majority of three. C becomes its leader for term 2. A remains the old leader in the minority partition, but its two acknowledgements are insufficient to commit new entries.

3. Why is a majority required?

Any two majorities overlap. Together with one vote per node per term and Raft's log-selection rules, this prevents separate partitions from independently establishing conflicting committed histories. If no partition contains a majority, none can commit new entries.

4. How does Raft preserve consistency during partitions?

Raft requires majority replication for new commitments and restricts voting to protect committed history. Minority partitions cannot commit independently. After communication resumes, higher terms supersede stale leadership, and log reconciliation repairs uncommitted differences.

Part8:



Explanation：

The leader initially contains five log entries, while the follower contains only three. The follower is missing entries 4 and 5, corresponding to commands D and E in the code. During recovery, the scenario appends these missing entries from the leader's log to the follower's log. The follower's log length increases from three to five.

This demonstrates how a lagging follower can catch up with the leader's history. However, equal log lengths alone do not prove identical contents. The scenario directly copies entries and does not implement network-based recovery or conflicting-entry reconciliation. The additional content comparison outputs "Logs identical: true", confirming that the leader and follower logs contain identical data after recovery.

Reflection Questions

1. Why does the follower trust the leader's history?

In full Raft, election and log-freshness rules ensure that an elected leader contains previously committed entries. Followers accept replication from a valid leader using term and log-consistency checks. This scenario assigns the leader directly and does not implement those checks.

2. Why are missing entries replayed?

Missing entries must be replicated in order so the follower can reconstruct the shared history. Committed entries can then be applied to its state machine in the same order as other nodes. This scenario only appends entries to the log; it does not execute their commands.

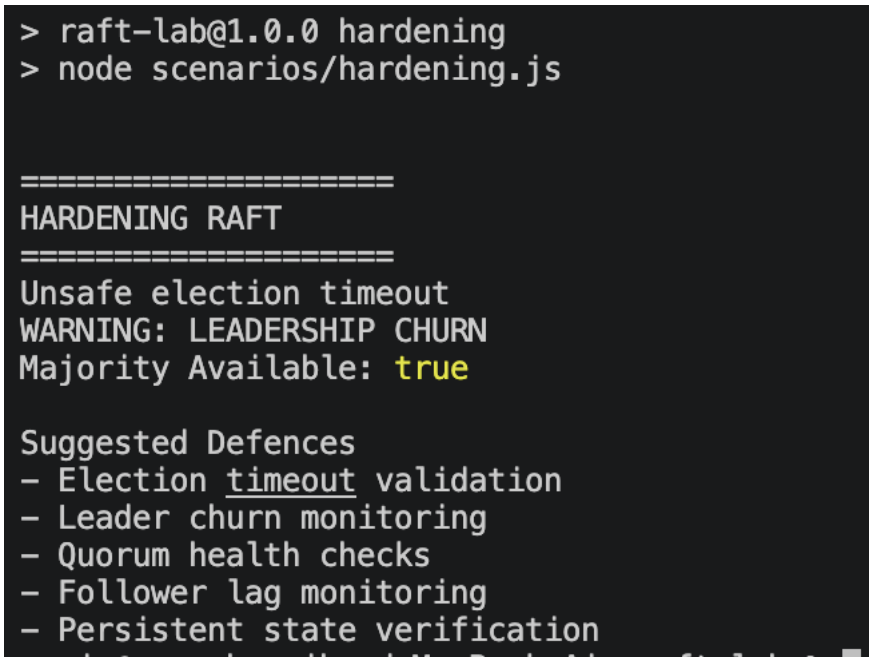
3. Why is log consistency important?

Consistent committed logs allow nodes with deterministic state machines to produce the same state from the same ordered commands. This supports reliable recovery and leadership changes without losing or contradicting committed operations.

4. What would happen if conflicting histories remained?

Unresolved conflicting histories could produce inconsistent application states if conflicting commands were applied. Full Raft detects log mismatches and replaces conflicting uncommitted entries with the leader's entries. This scenario handles missing entries only, so it does not demonstrate conflict resolution.

Part9：



Explanation：

The scenario checks operational conditions using fixed inputs. A timeout of 50 falls below the threshold of 100, triggering an unsafe-timeout message. An election count of 10 exceeds the threshold of five and produces a leadership-churn warning. Three online nodes out of five satisfy the majority requirement.

These checks illustrate timeout validation, leadership stability monitoring, and quorum availability. Their thresholds are illustrative, and they do not use live measurements. Follower-lag monitoring is demonstrated separately in Part 10: a gap of three entries triggers a warning, while recovery reduces the gap to zero. Persistent-state verification remains a suggested defence.

Reflection Questions

1. Why do election storms occur?

Election storms can occur when timeouts are too short relative to network delays or processing pauses. Followers repeatedly suspect leader failure and start elections. Packet loss and poorly coordinated timeout settings can further increase instability.

2. What is leadership churn?

Leadership churn means frequent changes of leader. Repeated elections interrupt normal replication and can reduce availability and throughput. This example checks a supplied election count, which does not directly measure actual leader changes over time.

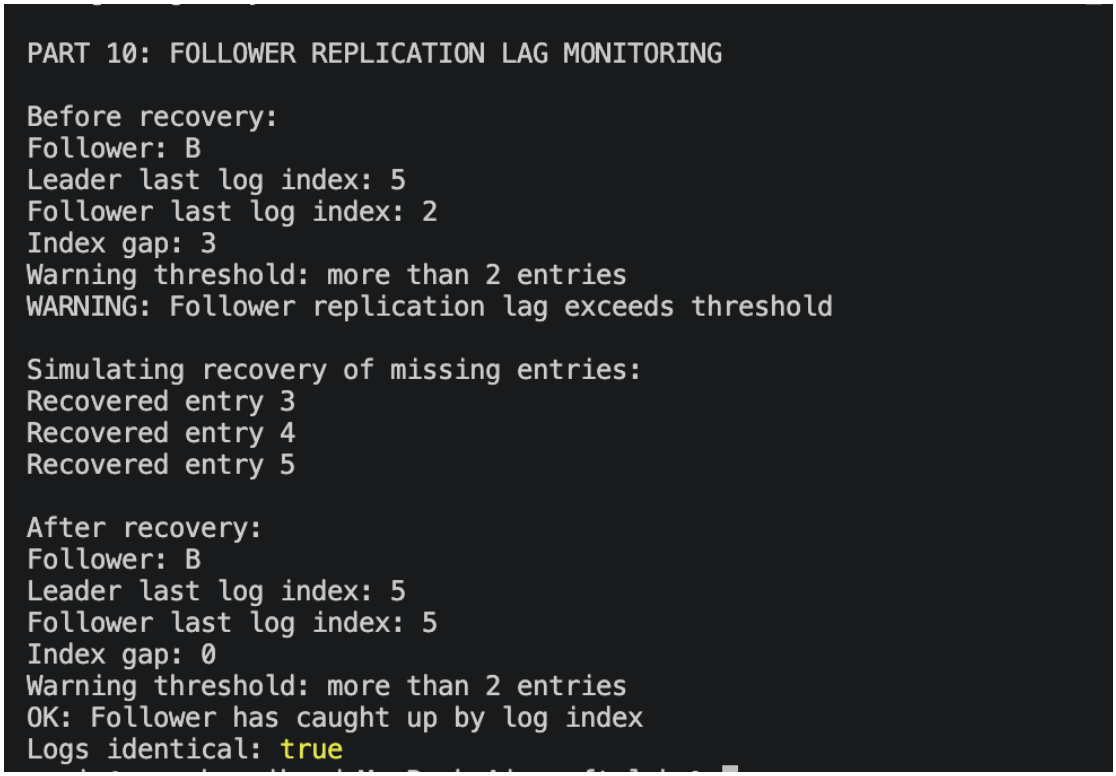
3. Why are slow followers a problem?

Slow followers fall behind the leader's log, increasing recovery time and reducing the number of up-to-date replicas. If too few nodes can acknowledge replication promptly, commitment slows or stops. One slow follower does not necessarily block a healthy majority.

4. Which operational assumptions are most critical?

Reliable progress requires a communicating majority, election timeouts suited to expected communication delays, and sufficient processing capacity. Correct persistent storage is also essential for preserving Raft's safety across restarts. This example checks only a small subset of these conditions.

Part10:



Explanation:

I added a follower replication-lag monitor that compares the leader's and follower's last log indexes. Initially, the leader's index is 5 and follower B's index is 2. The gap of three exceeds the configured threshold of two, triggering a warning. A separate recovery step copies entries 3, 4, and 5 to the follower. The next check reports a zero gap, and a content comparison confirms identical logs.

This enhancement makes replication lag visible and helps identify followers needing attention. However, it only detects lag; it does not automatically repair it. Equal indexes alone do not guarantee matching log contents.

Enhancement Details

1. Weakness being addressed

The original hardening scenario lists follower-lag monitoring as a suggested defence but does not implement it. Consequently, it provides no measurement or warning when a follower falls behind.

2. Assumption being protected

The enhancement monitors the operational expectation that followers can keep up with replication or eventually catch up. Persistent lag reduces redundancy and readiness for failures. Monitoring exposes this condition but does not itself guarantee recovery or consensus safety.

3. Implementation

The checkFollowerLag function calculates the difference between the leader's and follower's last log indexes. A gap greater than the configurable threshold triggers a warning. A zero gap reports that the follower has caught up by index. The function returns its measurements and status without modifying the logs.

4. Demonstration

Before recovery, the index gap is three, exceeding the threshold of two and triggering a warning. The demonstration separately copies the three missing entries. After recovery, the gap becomes zero. An independent content comparison outputs Logs identical: true, confirming matching logs in this example.

5. Remaining limitations

The monitor runs on demand and reads local, in-memory logs. It does not measure network delays, detect conflicting entries, or automatically repair followers. Equal indexes can hide different log contents. The recovery demonstration assumes a matching existing prefix, and the threshold requires adjustment for different workloads.

Reflection Questions

1. Which failures can be prevented?

The monitor does not directly prevent failures. Early warnings can help operators investigate and address persistent lag before it leaves the cluster with insufficient redundancy. Prevention depends on an appropriate response.

2. Which failures can only be detected?

The monitor detects excessive log-index lag and flags a follower whose index exceeds the leader's. It cannot determine whether the underlying cause is network loss, slow storage, or another issue.

3. How does the enhancement affect complexity?

Each check reads two final log indexes and compares them, requiring constant time per follower. Monitoring all followers scales linearly with follower count. Recovery and full content comparison are separate operations whose costs depend on the amount of data.

4. Does the enhancement introduce new assumptions?

It assumes valid, ordered logs, sufficiently current observations, and a threshold appropriate for the workload. The recovery demonstration additionally assumes that the follower's existing entries match the leader's prefix.

5. What additional mechanisms would be required in a production Raft deployment?

A production deployment needs durable state, correct term and voting rules, real replication acknowledgements, log-consistency checks, and crash recovery. Operational monitoring should include periodic measurements, reachability checks, time-based lag, and alert handling. Snapshot-based recovery may also be needed for severely lagging followers.